

EXP.NO: 3	AWK Scripting
DATE:	

AIM:

To study and implement AWK scripting for processing text files, performing mathematical calculations, and generating formatted outputs based on specific conditions.

ALGORITHM:

1. Initialization: Prepare an input data file named data.txt containing structured text with multiple fields (e.g., ID, Name, Salary, and Scores).
2. Input Reading: Read the input file line by line using the AWK utility.
3. Field Extraction: Use positional parameters (\$1, \$2, \$3, etc.) to access specific data columns and built-in variables like NR for row numbers and NF for the count of fields.
4. Summation Logic:
 - Initialize a accumulator variable (e.g., total = 0) in the BEGIN block.
 - Add the value of the target field (Salary) to the accumulator for every record processed.
 - Display the final sum in the END block after all lines are read.
5. Conditional Filtering: Apply if statements to check specific criteria, such as identifying employees with a salary greater than a defined threshold (e.g., 6000).
6. Complex Logic Implementation:
 - Evaluate multiple fields simultaneously using logical operators (e.g., || for OR).
 - Check if any subject score is below the passing mark (e.g., 45).
 - Assign and print a status of "Pass" or "Fail" based on the result of the logical evaluation.
7. Output Generation: Format and print the processed results to the standard output (terminal).

CODE:

```
101 Ravi 5000 5 60 55 70
102 Sita 7000 6 40 50 65
103 Arun 6500 3 80 85 90
104 Meena 8000 7 30 60 75
105 Kiran 5500 4 72 68 74
```

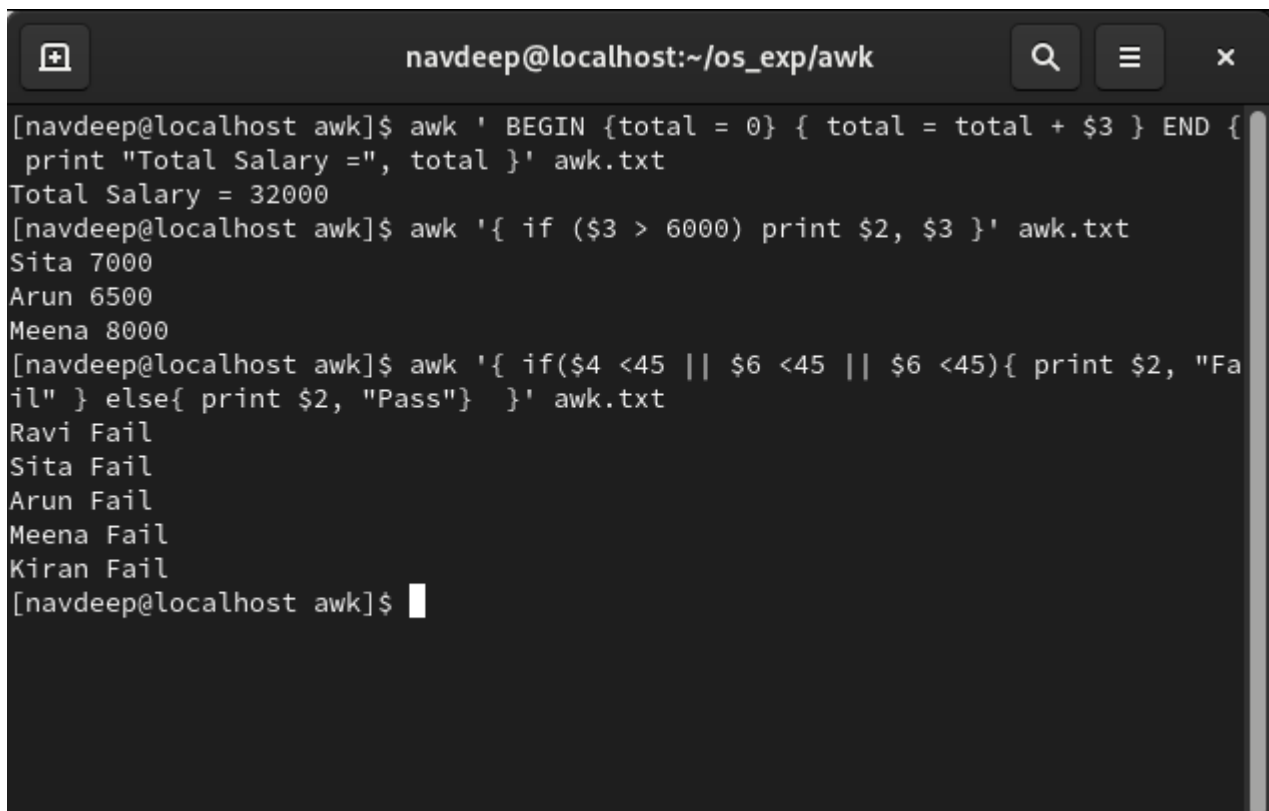
```
awk '
BEGIN {total = 0}
{
    total = total + $3
}
END {
```

```
    print "Total Salary =", total
}' data.txt
```

```
awk '{
    if ($3 > 6000)
        print $2, $3
}' data.txt
```

```
awk '{
    if ($5 < 45 || $6 < 45 || $7 < 45)
        print $2, "Fail"
    else
        print $2, "Pass"
}' data.txt
```

OUTPUT:

A terminal window titled 'navdeep@localhost:~/os_exp/awk' with search, menu, and close icons. It shows three AWK commands and their outputs. The first command calculates the total salary from 'awk.txt', resulting in 32000. The second command filters 'awk.txt' for salaries greater than 6000, listing Sita (7000), Arun (6500), and Meena (8000). The third command filters 'awk.txt' based on age (columns 5, 6, 7) to show 'Fail' or 'Pass' status, with all listed individuals (Ravi, Sita, Arun, Meena, Kiran) failing.

```
navdeep@localhost:~/os_exp/awk
[navdeep@localhost awk]$ awk ' BEGIN {total = 0} { total = total + $3 } END {
  print "Total Salary =", total }' awk.txt
Total Salary = 32000
[navdeep@localhost awk]$ awk '{ if ($3 > 6000) print $2, $3 }' awk.txt
Sita 7000
Arun 6500
Meena 8000
[navdeep@localhost awk]$ awk '{ if($4 <45 || $6 <45 || $6 <45){ print $2, "Fa
il" } else{ print $2, "Pass"} }' awk.txt
Ravi Fail
Sita Fail
Arun Fail
Meena Fail
Kiran Fail
[navdeep@localhost awk]$
```

RESULT:

Thus, AWK scripting was successfully implemented to process text data, perform arithmetic calculations, and apply logical conditions to generate meaningful and formatted output.